

Tutoría Fundamentos de Informática

Tipos de dato y variables en C++



Tipos de dato

- C++ ofrece distintos tipos para representar la información
- Cada uno ocupa un espacio diferente en memoria y permite realizar operaciones específicas

Estos son los más comunes que usaremos:

Tipo	Uso	Ejemplo
int	Números enteros sin decimales	10, 2000 o -123
float	Números decimales (baja precisión)	-17.26 o 100.78
double	Números decimales (alta precisión)	12.458 o 5.8908
char	Un solo carácter o entero pequeño (del 0 al 9)	'A' o '5'
string	Almacena una cadena de caracteres (texto)	"Hola mundo"
bool	Almacenan valores lógicos (verdadero o falso)	true (1) o false (0)

📌 **NOTA:** El tipo `float` se usa comúnmente para calcular porcentajes, precios y descuentos.

Diferencia: int vs char

Código	Tipo	¿Qué es?	Uso
<code>int n = 5;</code>	Número entero	Una cantidad (su valor es 5)	Operaciones matemáticas
<code>char n = '5';</code>	Carácter	Un símbolo (solo el dibujo del 5)	Representar letras, dígitos o símbolos

- Aunque ambos parezcan un **5** a simple vista, el programa los interpreta de forma completamente distinta:

5 → **int**

Valor **numérico**. Se puede operar matemáticamente: sumar, restar, multiplicar.

'5' → **char**

Carácter de **texto**. Se trata como un símbolo, no un número

Variables en C++



¿Qué es una variable?

Es un **espacio de memoria** donde almacenamos datos que podemos leer y modificar en cualquier momento del programa

- Piensa en una variable como una **caja con etiqueta**: el nombre la identifica de forma única y el tipo de dato determina qué puede guardar

Declaración de una variable

Para declarar una variable necesitamos dos elementos:

1 Tipo de dato

Determina qué valores puede guardar y qué operaciones podemos hacer con ella.

2 Identificador (nombre)

Nombre único que usaremos para acceder a la variable en el programa.

```
int edad;  
string nombre;  
float salario;
```

 **IMPORTANTE:** No podemos usar una variable que no ha sido inicializada, esto nos generaría un error

Tres formas de crear variables

1

Declaración

Sintaxis: tipo nombre;

```
int x; //declaracion
string n; // declaracion
float p; //declaracion
```

Crea la variable sin asignarle valor inicial

2

Inicialización

```
int x; //declaracion
x = 20; // inicializacion

string n; // declaracion
n = "hola mundo"; // inicializacion
```

Primero se declara y luego asigna un valor en una línea por aparte

3

Declaración + Inicialización

Sintaxis: tipo nombre = valor;

```
int j = 100; /* declaracion e
inicializacion */

cout << j << endl;
```

Declara y asigna un valor en el mismo paso

📌 **NOTA:** No podemos declarar dos variables con el mismo nombre dentro de un mismo ámbito.

Reglas para nombrar variables

Para crear variables necesitamos recordar estas reglas:

Regla	¿Qué está permitido?	Ejemplos válidos	Ejemplos no válidos
Primer caracter	Debe ser una letra (<code>a-z</code> , <code>A-Z</code>) o un guion bajo (<code>_</code>)	nota, id, X	1nota (Empieza con número)
Caracteres restantes	Letras, números y guión bajo (<code>_</code>)	calificacion1, totalSuma	precio\$, año (Símbolos no permitidos)
Espacios en blanco	No se permiten espacios	nombreUsuario	nombre usuario (Contiene un espacio)
Palabras reservadas	No usar palabras propias del lenguaje	entero, resultado	int, return (Son palabras pre-definidas)
Mayúsculas/minúsculas	El lenguaje es sensible a ellas, aunque se vean “iguales” son distintas	control y Control (Son variables distintas)	No aplica (Ambas son válidas, pero distintas)

Variables constantes (const)

Una constante es una variable cuyo valor **no puede cambiar** durante la ejecución del programa

Sintaxis

```
const tipo nombre = valor;
```

Ejemplos:

```
const float IVA = 0.13;
```

```
const char etiqueta = 'J';
```

 **IMPORTANTE:** Es obligatorio inicializar una constante en la misma línea de declaración. No se puede asignar después.

Ejercicio #1: Intercambio de variables

Desarrolla un programa en C++ que cumpla lo siguiente:

1. Declara dos variables enteras e inícialas con valores diferentes
2. Intercambia sus valores
3. Muestra los valores antes y después del intercambio



Ejemplo esperado en salida:

Antes: a = 5, b = 10

Después: a = 10, b = 5